LLD, also called Object-Oriented Design (OOD) or Software Architecture Design, converts high-level system requirements into actionable, clean, maintainable, and modular code. While High-Level Design (HLD) decides *which services, databases, and caches to use*, LLD defines the **internal classes, interfaces, relationships, design patterns, and database schemas** of an individual service.

Pillar	Focus	Core Concepts
Object-Oriented Programming (OOP)	Data and behavior encapsulation	Encapsulation, Abstraction, Inheritance, Polymorphism, Composition over Inheritance.
Design Principles	Rules for maintainable code	**SOLID** (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion), DRY, KISS, YAGNI.
Design Patterns (GoF)	Proven architectural templates	Creational (Factory, Singleton, Builder), Structural (Adapter, Decorator, Facade), Behavioral (Strategy, Observer, Command, State).
Schema & Data Modeling	Persisting internal application state	Normalization, primary/foreign keys, indexing strategies, join efficiency.
Concurrency & Thread Safety	Race condition prevention	Mutexes, Locks, Deadlocks, Atomic variables, Thread Pools, Producer-Consumer models.
UML Diagrams	Visual architectural modeling	Class Diagrams (relationships: association, aggregation, composition), Sequence Diagrams.

Key Pillars of LLDStep-by-Step LLD Roadmap (6–8 Weeks)

Phase 1: OOP Refresher & SOLID Principles (Weeks 1–2)

- Move beyond basic syntax to understand **Composition over Inheritance** (why extending classes leads to tight coupling and brittle code).
- Practice identifying SOLID violations in real codebases:
    - **Single Responsibility (SRP):** Splitting monolithic classes that handle both business logic and data access/serialization.
    - **Open/Closed (OCP):** Designing code so new features require adding new classes, not modifying existing if-else or switch blocks.
    - **Liskov Substitution (LSP):** Ensuring derived classes properly substitute their base types without throwing unexpected exceptions.
    - **Interface Segregation (ISP):** Splitting fat interfaces into smaller, role-specific interfaces.
    - **Dependency Inversion (DIP):** Injecting abstractions (interfaces) rather than instantiating concrete dependencies directly.

Phase 2: Core Design Patterns (Weeks 3–4)

Focus on the most frequent patterns used in production:

- **Strategy Pattern:** Swapping algorithms at runtime (e.g., dynamic payment processing via Stripe, PayPal, or Crypto).
- **Observer Pattern:** Event-driven notification models (e.g., updating multiple services when an order status changes).
- **Factory & Abstract Factory:** Decoupling object creation logic from business logic.
- **Decorator Pattern:** Adding responsibilities dynamically without subclassing (e.g., request logging or metric middleware wrappers).
- **State Pattern:** Managing complex object state transitions cleanly without nested conditions (e.g., Order statuses: Placed $\rightarrow$ Paid $\rightarrow$ Shipped $\rightarrow$ Delivered).

Phase 3: Concurrency, Thread Safety & State (Week 5)

- Implement thread-safe Singleton patterns and in-memory caches.
- Master synchronization primitives: mutexes, read-write locks, semaphores, and condition variables.
- Model race-condition-sensitive workflows (e.g., two users booking the exact same seat simultaneously).

Phase 4: Hands-On Machine Coding & Classical Problems (Weeks 6–8)

Implement working, testable code for standard LLD interview questions using an explicit 5-step framework:

1. Clarify requirements, functional use cases, and actors.
2. Identify core entities, attributes, and relationships.
3. Draw the UML Class Diagram (or define the core interfaces).
4. Apply relevant design patterns.
5. Write working, modular code with unit tests.

Must-Practice LLD Problems:

- **Parking Lot System:** Managing multiple vehicle sizes, spots, ticketing, and dynamic pricing strategies.
- **In-Memory Cache (LRU Cache):** Using a doubly linked list + hash map with eviction logic and concurrency locks.
- **Movie Ticket / Flight Booking (BookMyShow):** Seat allocation, locking mechanics, search, and concurrent reservation handling.
- **Elevator System:** Handling multiple requests, direction scheduling algorithms, and dispatch optimization.
- **Rate Limiter Library:** Implementing Token Bucket or Sliding Window algorithms as reusable middleware.

HLD

Domain	Key Topics to Master
Architectural Fundamentals	Latency vs. Throughput, Availability vs. Consistency, CAP Theorem, PACELC, Horizontal vs. Vertical Scaling.
Networking & Routing	DNS, CDN (Edge Caching), Reverse Proxies vs. Load Balancers (Layer 4 vs. Layer 7), WebSockets vs. SSE vs. Long Polling.
Data Storage & Scaling	SQL vs. NoSQL (document, key-value, wide-column), Indexing (B-Trees), Replication (Leader-Follower, Multi-Leader), Sharding/Partitioning strategies.
Caching Strategies	Redis/Memcached, Cache-aside, Write-through, Write-back, Cache Invalidation, Eviction policies (LRU, LFU), Thundering Herd problem.
Asynchronous Processing	Message Queues & Streaming (Kafka, RabbitMQ, SQS), Pub/Sub, Delivery Semantics (At-least-once, Exactly-once), Dead Letter Queues (DLQ).
Resilience & Security	Rate Limiting (Token Bucket, Sliding Window), Circuit Breakers, Retries with Exponential Backoff, Idempotency keys, JWT vs. Session Auth.

Low-Level Design (LLD)	SOLID principles, Clean Architecture, Design Patterns (Factory, Strategy, Observer, Decorator), Concurrency & Thread Safety.

4-Phase Learning Roadmap (12–16 Weeks)

Phase 1: Deepening the Backend Foundation (Weeks 1–3)

- Shift focus from writing ORM queries to database internals: index lookups, connection pooling, and ACID guarantees.
- Master API design subtleties: Cursor-based vs. offset pagination, idempotent POST/PUT endpoints, and rate-limiting patterns.
- Practice basic math: Back-of-the-envelope calculations (converting DAU to requests/sec, estimating storage over 5 years).

Phase 2: Distributed System Building Blocks (Weeks 4–7)

- Study caching patterns in depth: when to use read-through vs. cache-aside, and how cache stamps occur.
- Learn decoupling techniques using queues: background job processing, event-driven workflows, and buffer leveling.
- Understand database partitioning: Consistent hashing, range-based vs. hash-based sharding.

Phase 3: Classic System Design Blueprints (Weeks 8–12)

Analyze standard problems using a structured approach (Requirements $\rightarrow$ Capacity Estimation $\rightarrow$ API & Data Model $\rightarrow$ High-Level Diagram $\rightarrow$ Bottleneck Deep-Dive):

- **Read-Heavy System:** URL Shortener (e.g., TinyURL), Pastebin.
- **Write-Heavy / Analytics System:** Distributed Counter, Metrics/Logging Collector.
- **Real-Time System:** 1:1 and Group Chat App (WhatsApp/Slack).
- **Feed / Fanout System:** Twitter/Instagram feed generation (Push vs. Pull model).
- **Storage / Blob Service:** File hosting system (Google Drive / Dropbox) with chunking.

Phase 4: Low-Level Architecture & Interview Prep (Weeks 13–16)

- Practice refactoring monolithic code into modular layers using SOLID principles.
- Design classic LLD problems: Parking Lot, Rate Limiter library, LRU Cache, Movie Ticket Booking (handling race conditions during seat locking).
- Do mock design reviews: focus on justifying **trade-offs** (e.g., why chose MongoDB over PostgreSQL, or why favored availability over strong consistency) rather than just listing technologies.

Area	Key Concepts to Master
Data Modeling & Schema	Relational Algebra, Normalization (1NF through BCNF), Denormalization trade-offs, ER modeling, foreign keys, and constraint enforcement.
Storage Engines & Disk Layout	Pages/Blocks, Row-oriented vs. Columnar storage, Buffer Pool management, Write-Ahead Logging (WAL) for durability.
Indexing Mechanics	Clustered vs. Non-Clustered indexes, B-Trees / B^+ Trees (structure and node splits), Hash Indexes, LSM-Trees (Log-Structured Merge-trees for write-heavy engines like Cassandra/RocksDB).
Transactions & Concurrency	ACID semantics, Isolation Levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable), Read phenomena (Dirty Read, Non-Repeatable Read, Phantom Read).
Concurrency Control & Locking	Two-Phase Locking (2PL), Pessimistic vs. Optimistic Concurrency Control (OCC), Deadlock detection and prevention, Multi-Version Concurrency Control (MVCC).
Query Engine & Optimization	SQL Parsing, Query Optimization (Cost-Based vs. Rule-Based), Execution Plans (EXPLAIN ANALYZE), Sequential Scans vs. Index Scans, Join algorithms (Nested Loop, Hash Join, Merge Join).
Distributed Databases & Scale	Replication (Sync vs. Async, Read Replicas), Sharding strategies, Distributed Transactions (Two-Phase Commit / 2PC), Consensus algorithms (Raft, Paxos).

DBMS is about mastering how data is physically stored, indexed, locked, and retrieved across single-node and distributed systems.

6-Week Practical DBMS Roadmap

Week 1: Advanced Relational Modeling & Schema Design

- Move beyond basic ORM models: design schemas that balance query ergonomics against write duplication.
- Understand normalization (up to 3NF) for transactional integrity and when to intentionally denormalize for read-heavy systems.
- Deep dive into PostgreSQL data types: JSONB vs. traditional relational columns, UUID vs. Auto-increment integer performance.

Week 2: Storage Engines & The Memory Hierarchy

- Study how databases avoid disk I/O bottlenecks: in-memory Buffer Pools, LRU page replacement, and dirty page flushing.
- Learn the mechanics of the Write-Ahead Log (WAL): why sequential disk writes make crash recovery possible without sacrificing throughput.
- Compare Row-oriented storage (PostgreSQL/MySQL for OLTP) with Columnar storage (ClickHouse/Snowflake for analytics/OLAP).

Week 3: Indexing Under the Hood

- Study the internal mechanics of B^+ Trees: depth, fanout, and range-scan efficiency.
- Analyze composite index ordering: the "Leftmost Prefix Rule" and how incorrect column order causes full table scans.
- Understand write amplification: how adding secondary indexes speeds up SELECT queries while slowing down INSERT, UPDATE, and DELETE.

Week 4: Transactions, Isolation Levels & MVCC

- Run hands-on tests in two separate SQL shells using BEGIN TRANSACTION to observe dirty reads, non-repeatable reads, and phantoms under each isolation level.
- Study MVCC (Multi-Version Concurrency Control): how PostgreSQL and MySQL InnoDB allow readers to not block writers, and writers to not block readers.
- Practice pessimistic locks (SELECT ... FOR UPDATE) and optimistic concurrency control (using a version column).

Week 5: Query Planning & Performance Tuning

- Learn to read execution plans: EXPLAIN (ANALYZE, BUFFERS) in Postgres or EXPLAIN in MySQL.
- Identify common bottlenecks: missing indexes, implicit type conversions invalidating indexes, and N+1 query patterns generated by full-stack ORMs.
- Tune join performance by understanding when the engine selects Nested Loop, Hash Join, or Sort-Merge Join.

Week 6: Scaling & Distributed Databases

- Set up primary-replica replication and handle replication lag in production architectures.
- Implement horizontal database sharding: consistent hashing, cross-shard joins, and distributed unique ID generation (e.g., Snowflake IDs).
- Compare relational databases with specialized stores: Key-Value (Redis), Document (MongoDB), and Time-Series (TimescaleDB).